

PLANO DE TRABALHO E O ORGANIZAÇÃO DA APRESENTAÇÃO

A apresentação seguirá uma única linha de raciocínio: problema e contexto → conceitos e funcionamento → aplicações e estudo de caso → demonstração → limitações e tendências → síntese da literatura e referências.

1. Pedro – Contextualização, conceitos e funcionamento. Esta parte apresenta o problema, introduz Engenharia de Plataforma, DevEx, Self-Service, Golden Paths e IDP e explica, com base na literatura científica utilizada no trabalho, como esses elementos se relacionam.
2. Ronaldo – Aplicações, estudo de caso e demonstração. Esta parte parte dos conceitos apresentados por Pedro e mostra como eles aparecem nas aplicações organizacionais, no estudo de caso de Internal Developer Platform e na demonstração prática já descrita no trabalho.
3. Emillyn – Limitações, tendências, organização dos materiais e referências. Esta parte retoma a plataforma apresentada por Ronaldo, discute limitações e lacunas da literatura, apresenta tendências identificadas na pesquisa e encerra com a organização da literatura e as referências.

ATIVIDADE PRÁTICA – ETAPA 1

Três conceitos associados ao tema:

Developer Experience (DevEx), Self-Service e Golden Paths.

Duas aplicações possíveis:

Catálogo de serviços e autosserviço para tarefas de infraestrutura.

Dúvida inicial:

Em quais situações uma plataforma interna não deve ser utilizada?

Um risco/limitação já identificado no trabalho: uma plataforma interna pode aumentar a complexidade se for criada de forma muito complexa.

Organização relacionada ao tema:

Tampere University, instituição vinculada ao estudo de caso de Veikko Nieminen.

Problema:

O desenvolvimento de muitos programadores no momento atual está sendo afetado por uma crise: um excesso de esforço cognitivo, especificamente, o esforço que é necessário para um desenvolvedor manter o seu desenvolvimento em progresso, sendo interrompido por dificuldades em sua plataforma como as limitações dela e a manutenção de funções desatualizadas ou incompatíveis, com isso, foi considerado as possíveis alternativas para diminuir essa carga cognitiva e como modificar as plataformas de desenvolvimento utilizadas.

<https://ijctjournal.org/platform-engineering-internal-developer/>

Introdução do tópico:

A solução sugerida é a engenharia de plataforma, em qual é aplicado o **desenvolvimento interno da plataforma**, onde ela é modificada e desenvolvida a fim de melhorar a **experiência do desenvolvedor**, isso pode ser alcançável por meio dos **Golden Paths e Self-Service**, onde o desenvolvedor não é dependente da equipe de suporte que

normalmente iria forçar um bottleneck para o desenvolvedor (**Ops de Tickets**) , ao invés disso, sendo capaz de abstrair o problema de forma fácil de utilizar, sem ter a necessidade de gastar esforço cognitivo desnecessariamente em plataformas desatualizadas.

<https://www.alura.com.br/artigos/platform-engineering>

Essa terminologia vai ser elaborada conforme a progressão, pois elas servem tanto como métrica de efetividade quanto um guia de metodologia para o uso apropriado de engenharia de plataforma.

Experiência de desenvolvedor

Para aplicar o conceito, é necessário identificar as dificuldades causadas. De acordo com uma enquete de 65,000 desenvolvedores globalmente, foi confirmado que a satisfação do desenvolvedor sobre ferramentas internas está entre os piores resultados, então se sabe que a melhora de ferramentas internas é o foco das melhorias. Para isso, é examinado a **experiência do desenvolvedor**, que identifica a rotina, produção e fluxo de trabalho do desenvolvedor. partir dessas informações, é possível ver os bottlenecks de criação causado por ferramentas internas insatisfatórias, e o impacto das ferramentas de engenharia de plataforma.

<https://getdx.com/blog/developer-experience/>

<https://survey.stackoverflow.co/2024/>

4- Self Service:

O **self service** trabalha a partir disso, abstraindo os problemas que interrompem o feedback loops de um programador e criando uma experiência com menos esforço cognitivo. Então se ele quiser conectar ferramentas diferentes, pesquisar documentação, ou customizar a sua experiência de acordo com a especialização desejada, ele vai ser capaz de realizar a fim do framework de **self service**.

<https://ijctjournal.org/platform-engineering-internal-developer/>

5- Golden Paths:

Os **Golden Paths** serão os guias da configuração da plataforma, com a ideia sendo que um **Golden Path** oferece a sequência de ações e eventos mais lisa e cooperativa para o **Developer Experience** do programador, criando um streamlining das ações desejadas e convenientes para um desenvolvedor ter a melhor experiência. Ele serve como um corrimão para se apoiar, somente tendo que usar **self-service** para situações que necessitam de configurações mais especializadas - ele não deve ser tratado como um portão, restringindo e forçando o desenvolvedor ao andar ao redor dele ou entrar em conflito com suas configurações.

[https://www.frontiersin.org/journals/computer-science/articles/10.3389/fcomp.2026.1814498/f](https://www.frontiersin.org/journals/computer-science/articles/10.3389/fcomp.2026.1814498/full)

ull

COMO A TECNOLOGIA FUNCIONA?

Os artigos científicos utilizados no trabalho tratam a Internal Developer Platform (IDP) como uma integração de ferramentas e processos voltada a simplificar o ciclo de desenvolvimento. Ciancarini et al. (2025) descrevem uma IDP auto-hospedada e de código aberto baseada em arquitetura de microsserviços, com o Compositional Agile System (CAS) como implementação proposta.

Na literatura sobre Developer Experience, Greiler, Storey e Noda (2023) apresentam um framework construído a partir de entrevistas semiestruturadas com 21 desenvolvedores da indústria para compreender fatores que afetam a experiência do desenvolvedor. Essa evidência sustenta a ligação, já presente no trabalho, entre a experiência do desenvolvedor e a organização das ferramentas e processos.

Assim, a plataforma integra capacidades e ferramentas, o desenvolvedor acessa recursos por mecanismos de autosserviço e caminhos preparados, e a equipe de plataforma mantém essa camada como um produto interno.

ARTIGO CIENTÍFICO

The Design and Realization of a Self-Hosted and Open-Source Agile Internal Development Platform.

O artigo parte do desafio de integrar tecnologias e ferramentas para apoiar o desenvolvimento e o gerenciamento de projetos de software, especialmente em contextos de desenvolvimento ágil.

Os autores apresentam o Compositional Agile System (CAS), uma Internal Development Platform extensível, baseada em arquitetura de microsserviços, com características de código aberto e auto-hospedagem.

Descreve a realização da plataforma e cenários de uso, incluindo experiências anteriores relacionadas à Administração Pública italiana e ao ensino remoto de Engenharia de Software.

Aplicações da Engenharia de Plataformas

A Engenharia de Plataformas pode ser aplicada dentro das organizações para facilitar o trabalho dos desenvolvedores e reduzir a complexidade causada pelo uso de muitas ferramentas, processos e ambientes diferentes. Uma das formas mais comuns de aplicação é por meio de uma Internal Developer Platform (IDP), ou plataforma interna para desenvolvedores. Essa plataforma funciona como um ponto central onde o desenvolvedor pode acessar recursos, serviços, documentações e automações necessárias para o desenvolvimento de software. O objetivo principal é diminuir tarefas manuais, reduzir a dependência constante de equipes de infraestrutura e facilitar o acesso às ferramentas utilizadas no dia a dia. Entre as aplicações mais comuns estão o catálogo de serviços, o self-service, os Golden Paths, a automação de processos e a integração de ferramentas.

Catálogo de Serviços

O catálogo de serviços permite organizar informações sobre sistemas, APIs, equipes responsáveis, repositórios, documentação e outros recursos importantes. Esse tipo de

recurso é útil principalmente em empresas que possuem muitos sistemas e microsserviços, pois as informações podem acabar espalhadas em várias ferramentas diferentes.

Com um catálogo central o desenvolvedor consegue localizar com mais facilidade quem é responsável por determinado serviço, onde está o repositório, onde encontrar a documentação e quais recursos estão relacionados ao sistema. Dessa forma, a plataforma melhora a organização e facilita o entendimento do ambiente tecnológico da empresa.

Self-Service

O self-service permite que o desenvolvedor execute determinadas tarefas sem precisar abrir chamados ou esperar que outra equipe faça todo o processo manualmente. Um exemplo seria a criação de um banco de dados, de um ambiente de testes ou de um novo serviço. Em um modelo tradicional, o desenvolvedor poderia depender da equipe de infraestrutura para executar essas tarefas, já com o self-service, essas ações podem ficar disponíveis diretamente na plataforma, permitindo que o próprio desenvolvedor inicie o processo. A automação realiza os passos necessários seguindo as regras definidas pela organização, esse modelo pode reduzir atrasos e diminuir a quantidade de tarefas repetitivas ou processos lentos realizados pelas equipes de infraestrutura.

Golden Paths

Os Golden Paths são caminhos recomendados e preparados pela equipe de plataforma para tarefas comuns do desenvolvimento. Um exemplo seria a criação de um novo microsserviço em Java com Spring Boot. Nesse caso a plataforma pode oferecer um modelo que já contenha a estrutura inicial do projeto, o repositório Git, o dockerfile e outras configurações básicas. Dessa forma, o desenvolvedor não precisa iniciar tudo do zero e pode seguir um padrão já utilizado pela empresa.

O objetivo do Golden Path não é obrigar todas as equipes a trabalharem exatamente da mesma forma, mas oferecer um caminho pronto e confiável para os casos mais comuns, esses caminhos devem funcionar como facilitadores do desenvolvimento e não como regras obrigatórias que impeçam outras soluções.

Developer Experience

A Engenharia de Plataformas também está relacionada com a Developer Experience (DevEx). A DevEx representa a experiência do desenvolvedor com as ferramentas, processos e ambientes utilizados durante o trabalho. Quando existem muitas ferramentas separadas, documentações espalhadas e processos manuais, o desenvolvedor pode gastar muito tempo resolvendo problemas de ambiente ou procurando informações. Isso pode aumentar a dificuldade do trabalho e diminuir a produtividade. Greiler, Storey e Noda (2023) mostram que a experiência do desenvolvedor está relacionada a fatores como produtividade, satisfação, engajamento e eficiência no trabalho. O estudo foi desenvolvido a partir de entrevistas com profissionais da indústria e propôs um framework voltado à compreensão e melhoria da experiência dos desenvolvedores. Nesse sentido, a Engenharia de Plataformas busca melhorar a DevEx por meio da integração de ferramentas, automação, centralização de informações e padronização de processos.

Estudo de Caso: Internal Developer Platform

Um estudo importante sobre Engenharia de Plataformas foi desenvolvido por Veikko Nieminen, em 2024, na Tampere University. O trabalho, chamado Internal Developer Platform, analisa a aplicação desse conceito dentro de uma organização e discute problemas encontrados no ambiente de desenvolvimento (NIEMINEN, 2024).

O estudo apresenta um cenário com diferentes ferramentas, equipes e formas de trabalho. Entre os problemas observados estão dificuldades relacionadas à visibilidade dos sistemas, ao onboarding de novos profissionais e à falta de padronização entre as equipes. Essas situações ajudam a mostrar por que uma plataforma interna pode ser útil em organizações maiores, principalmente quando o ambiente tecnológico começa a crescer e se tornar mais difícil de administrar.

A proposta analisada envolve a utilização de uma Internal Developer Platform como uma camada central para integrar diferentes ferramentas e processos. A ideia não é necessariamente substituir todas as tecnologias que a empresa já utiliza. A plataforma pode funcionar como uma camada de acesso que conecta recursos existentes e apresenta essas capacidades de uma maneira mais simples para os desenvolvedores. Assim, informações sobre serviços, documentação, repositórios, pipelines, infraestrutura e equipes responsáveis podem ser acessadas de maneira mais centralizada.

Benefícios da Plataforma

Entre os principais benefícios esperados estão uma melhor visibilidade dos sistemas, melhoria no onboarding, maior padronização e automação de tarefas. O onboarding pode ser facilitado porque novos desenvolvedores passam a ter um ponto central para entender os sistemas e ferramentas da organização. Em vez de descobrir cada ferramenta separadamente, a plataforma pode apresentar as principais informações e recursos necessários para iniciar o trabalho. A padronização também pode ser aplicada por meio de templates e Golden Paths, permitindo que novos projetos sejam criados seguindo práticas definidas pela organização. A automação pode reduzir tarefas repetitivas, como criação de ambientes, configuração de pipelines e provisionamento de recursos. Dessa forma, a equipe de infraestrutura pode dedicar mais tempo a atividades mais complexas, enquanto tarefas comuns ficam disponíveis por meio da plataforma.

Limitações e Cuidados

Uma plataforma interna precisa ser simples e realmente resolver problemas dos desenvolvedores, caso seja criada de forma muito complexa, ela pode se tornar apenas mais uma ferramenta difícil de utilizar. Nesse caso, em vez de reduzir a carga de trabalho, a plataforma pode aumentar a complexidade.

A revisão de Anjum (2026) destaca que ainda existe uma diferença significativa entre a quantidade de conhecimento produzido pela indústria e a quantidade de estudos acadêmicos sobre Engenharia de Plataformas. O autor também destaca a necessidade de mais estudos empíricos sobre os resultados dessas plataformas nas organizações.

Demonstração Prática

Para a demonstração prática foi desenvolvida uma plataforma ilustrativa. Ela foi criada com o intuito de apresentar, de forma prática, como um portal interno de engenharia funcionaria em um ambiente real. Simula funcionalidades e processos que facilitam o dia a dia das equipes de desenvolvimento, como acompanhamento de serviços, consulta de indicadores, acesso à documentação e execução de tarefas de infraestrutura por autosserviço.

Visão geral da plataforma

A plataforma é um portal interno de engenharia criado para centralizar os principais recursos necessários no desenvolvimento, na operação e no acompanhamento de aplicações.

Ela reúne informações e funcionalidades que normalmente ficariam distribuídas entre diferentes ferramentas, como documentação, infraestrutura, monitoramento e solicitações operacionais.

O que a plataforma realiza

- Centraliza a visualização dos serviços e aplicações da empresa.
- Exibe indicadores de saúde, disponibilidade e desempenho dos sistemas.
- Disponibiliza documentação técnica e orientações de uso.
- Oferece recursos de autosserviço para tarefas de infraestrutura, como a criação de bancos de dados PostgreSQL.
- Permite acompanhar solicitações e processos executados pela própria plataforma.
- Facilita o acesso a informações relevantes para desenvolvimento, manutenção e operação dos sistemas.

Intuito da plataforma

O principal objetivo é oferecer mais autonomia às equipes de desenvolvimento. Em vez de depender de chamados manuais ou da intervenção constante da equipe de infraestrutura, os desenvolvedores podem executar tarefas padronizadas diretamente pela plataforma.

Com isso, a plataforma busca:

- Reduzir burocracias e tempo gasto em processos operacionais.
- Padronizar a criação e a gestão de recursos técnicos.
- Diminuir erros causados por processos manuais.
- Acelerar a entrega e manutenção de aplicações.
- Melhorar a visibilidade sobre os serviços e sua infraestrutura.
- Aproximar desenvolvimento, operações e governança em um único ambiente.

Limitações e lacunas na literatura

A literatura atual destaca várias lacunas conceituais, metodológicas e empíricas em *Engenharia de Plataforma*:

- **Definições e terminologia inconsistentes:** Há falta de consenso sobre o que exatamente constitui “engenharia de plataforma”. Estudos de revisão apontam que o campo ainda é evolutivo, com termos mal padronizados. Por exemplo, Pitkar (2026) observa que “a disciplina [de platform engineering] continua em evolução, com pesquisa empírica limitada e terminologia inconsistente”. Essa ambiguidade complica comparações entre estudos.
- **Escassez de pesquisa empírica:** Há poucas pesquisas quantitativas controladas. Anjum (2026) constatou que apenas 2,3% (2 de 88) das fontes em seu levantamento eram de periódicos de alto impacto, e praticamente nenhuma evidência empírica revisada por pares sustenta práticas comuns (como scorecards de plataforma). Oliveira et al. (2026) explicitam que “há uma lacuna notável em estudos empíricos” comparando IDPs com processos manuais, indicando ausência de dados concretos nos cenários reais. Assim, grande parte das recomendações provém de relatórios da indústria ou estudos de caso pontuais, limitando a validade estatística.
- **Amostras e generalização limitadas:** Quando há estudos de campo (por exemplo Seknametla & Sunkara 2023), suas amostras são pequenas ou específicas de poucas organizações. Por exemplo, Seknametla (2023) analisou dados de apenas três empresas, de forma a exemplificar o framework proposto. Embora os resultados indiquem benefícios, não garantem generalização ampla. Esta limitação é mencionada indiretamente ao final do estudo, ao sugerir estudos longitudinais e mais casos para validar os resultados.
- **Foco em literatura industrial:** Vários levantamentos (como Anjum 2026) mostram que a maior parte do conhecimento vem da literatura cinzenta (blogs, relatórios, white papers) e comunidades de praticantes. Por isso, conceitos-chave muitas vezes dependem de posicionamentos de fornecedores (por exemplo, Gartner, CNCF) e não de análise acadêmica rigorosa. Essa dependência pode introduzir viés comercial e falta de criticidade.
- **Falta de métricas e modelos consolidados:** Os trabalhos destacam carência de ferramentas padronizadas de mensuração. Anjum (2026) relata a inexistência de métricas formais avaliando scorecards de portais internos de desenvolvedores, apesar de sua adoção comercial generalizada. Pitkar (2026) e Anjum (2026) sugerem desenvolvimento de maturidade de plataformas e métricas específicas (além das tradicionais DORA/SPACE) como prioridades de pesquisa futura. Até o momento, não há consenso nem validação empírica de tais modelos, o que constitui importante lacuna metodológica.

2. Tendências e direções futuras

A literatura recente aponta várias direções emergentes para Engenharia de Plataforma:

- **Plataformas orientadas a observabilidade e dados:** Tendência em concentrar métricas e monitoramento na plataforma.
- **Aplicação de inteligência artificial:** Vários autores sugerem que IA/ML serão incorporados em IDPs. Pitkar cita “AI-infused IDPs” como tendência futura.
- **Plataforma como produto (“product mindset”):** Há consenso de que tratar a plataforma interna como um produto é bem-sucedido. Pitkar (2026) destaca que “considerar plataformas como produtos” é fator-chave de sucesso.
- **Modelos de referência e frameworks formais:** Vários trabalhos propõem ou sugerem formalização. Van de Kamp et al. (2024) introduziram um *Modelo de Referência de Engenharia de Plataforma (PE-RM)* baseado no framework RM-ODP, para dar uma “linguagem comum” e roadmap de implementação.
- **Novas funções e habilidades:** Observa-se a ascensão de um novo papel: o *engenheiro de plataforma*. Pitkar comenta sobre “novos caminhos de carreira emergentes”, onde organizações precisam distinguir engenheiros de plataforma de engenheiros DevOps tradicionais.
- **Ênfase em DevEx e medições qualitativas:** Além de métricas clássicas (DORA/SPACE), a tendência é incluir pesquisas qualitativas contínuas de satisfação e carga cognitiva.
- Em resumo, os autores projetam um futuro com **IDPs cada vez mais inteligentes (observabilidade+IA), plataformas tratadas como produtos** com cultura de DevEx, e o desenvolvimento de **modelos e métricas formais**.

Organização da literatura por tema, método e evidência

Para facilitar a análise, os materiais foram agrupados em três dimensões principais:

- **Por tema/conceito central:**

Plataforma Interna para Desenvolvedores (IDP): Foco em estudos que definem IDPs, descrevem seus componentes (portais, catálogos, pipelines) e discutem benefícios. Exemplos: Pitkar (2026) discute IDPs abstraindo infraestrutura.

Experiência do Desenvolvedor (DevEx): Estudos que medem ou discutem como as plataformas melhoram a satisfação, fluxo e produtividade dos devs. Pitkar foca em DevEx como resultado da plataforma.

Self-Service / Automação: Artigos que enfatizam capacidades de autoatendimento. Os trabalhos analisados destacam interfaces self-service. Por exemplo, Guisao et al. (2026) cita “capacidades self-service” e automação como alicerces.

Métricas DORA/SPACE e métricas de DevEx: Vários trabalhos incorporam métricas de entrega contínua (tempo de deploy, frequência etc.) ou frameworks de produtividade (SPACE) para avaliar plataformas.

Plataforma-como-Produto: Conceito em que a plataforma interna é tratada como produto com ciclo de vida próprio. Destacado por Pitkar (2026) e Anjum (2026) como prática recomendada. Estudos teóricos e cases de indústria reforçam equipe dedicada, feedback dos usuários da plataforma etc.

- **Por método de pesquisa:**

Revisões sistemáticas (MR): Cobrindo e sintetizando literatura existente, geralmente combinando periódicos e fontes de indústria. Exemplos: Guisao et al. (2026) – mapeamento sistemático (Systematic Mapping); Pitkar (2026) – revisão sistemática de literatura.

Estudos empíricos/casuísticos: Pesquisas de campo, experimentos controlados ou estudo de caso. Oliveira et al. (2026) – experimento comparativo (IDP vs manual) com 8 participantes; Allam (2022) – análise de caso comparativo de empresas usando PEaaS. Esses fornecem evidência prática dos impactos (mesmo que amostras sejam pequenas).

Modelos conceituais e proposições: Trabalhos que propõem frameworks ou modelos de referência. Ex.: van de Kamp et al. (2024) – Modelo Referência usando RM-ODP; e o artigo “AI-driven platform engineering” (Muggalla 2026) que propõe arquitetura conceitual APEXA. São estudos teóricos sem dados empíricos, mas que ajudam a estruturar a discussão.

Conceitos centrais: IDP, DevEx, Self-Service/Automação

Três conceitos-chave se repetem como fundamentos da Engenharia de Plataforma:

- **Internal Developer Platform (IDP):** Uma plataforma interna unificada de ferramentas e serviços para desenvolvedores, visando abstrair a complexidade infraestrutural. Vários autores definem IDP assim. Por exemplo, Pitkar (2026) afirma que “IDPs ocultam a complexidade da infraestrutura e habilitam self-service” para melhorar o DevEx. Guisao et al. (2026) consideram IDPs como artefatos essenciais da engenharia de plataforma, com módulos de provisionamento, pipelines e portais como componentes básicos. Em suma, IDPs são o *produto central* das iniciativas de platform engineering.
- **Developer Experience (DevEx):** Engloba a satisfação, eficiência e facilidade de uso que um desenvolvedor tem ao usar a plataforma. Em essência, é a **meta** da engenharia de plataforma. Estudos enfatizam que melhores DevEx levam a produtividade maior. Pitkar resume: “o uso de IDPs pode melhorar significativamente a experiência dos desenvolvedores, reduzindo a carga cognitiva e melhorando fluxo e satisfação”. Guisao destaca que platform engineering integra Dev e Ops “para aprimorar a experiência do desenvolvedor” e acelerar entregas. Assim, DevEx aparece em quase toda a literatura como *motivo central* para adotar plataformas internas.
- **Self-Service e Automação:** Capacidade de os desenvolvedores executarem tarefas comuns sem esperar por intervenção da equipe de infraestrutura. Esse é o *mecanismo* pelo qual IDPs geram DevEx. Guisao et al. mencionam explicitamente “capacidades self-service” e “infraestrutura automatizada” como características-chave que “melhoram a eficiência do desenvolvimento”. Esses elementos permitem redução de sobrecarga cognitiva: ao delegar tarefas de configuração à plataforma, os devs gastam menos tempo com operações básicas.

Tabela comparativa dos principais artigos

A tabela abaixo resume características-chave dos trabalhos prioritários (autor, objetivo, método, dados/ambiente, achados e limitações).

Estudo (autor, ano)	Objetivo	Método	Amostra / Dados	Principais achados	Limitações
Guisao <i>et al.</i> (2026)	Mapear o estado da arte da plataforma eng. como evolução do DevOps.	Systematic Mapping (mapeamento sistemático)	Revisão de literatura (poucos artigos peer-reviewed sobre o tema)	Define platform eng. como evolução do DevOps, integra Dev/Ops para melhorar DevEx. Destaca IDPs, self-service, automação como componentes chave. Relaciona plataforma com aumento de eficiência e escalabilidade.	Pouca literatura disponível; limitações não detalhadas no artigo (campo muito novo). Dados secundários dependentes da literatura incipiente.
Pitkar (2026)	Revisão sistemática sobre plataforma eng. e DevEx.	Revisão Sistemática da Literatura	~20–30 fontes (artigos, relatórios, blogs) selecionados (método misto)	Conclui-se que IDPs bem implementados reduzem carga cognitiva e aceleram entregas. O sucesso requer “mindset de produto” (golden paths, portais, padronização flexível). Desafia: níveis de abstração,	Pesquisa secundária, escopo de fontes variado (inclui relatórios e blogs), pouco foco em evidência empírica direta. Lacunas: falta de estudos de campo e terminologia unificada.

				barreiras organizacionais, sustentar plataformas. Tendências futuras: observabilidade, IA, modelos de referência.	
Anjum (2026)	MLR de platform eng. e portais internos (inclui literatura cinzenta).	Revisão Multivocal (MLR) em 5 BD acadêmicas + fontes cinzas	88 fontes (academic + gray literature, categorizadas em tiers)	Identifica taxonomia de componentes de IDPs a partir de 36 fontes (Figura abaixo)	Contribui com síntese ampla, mas dependente de fontes não revisadas. Exclui muitos estudos acadêmicos se aplicado somente a peer-review. Não avalia empiricamente nenhuma plataforma.
Sekname tla & Sunkara (2023)	Propor framework para medir redução de carga cognitiva e produtividade com IDPs.	Estudo de caso / análise de dados em 3 organizações + framework teórico (CLP).	3 empresas (não especificadas), dados de entrevistas/pesquisas + métricas DORA/SPACE	Constatam que IDPs bem feitas aumentam a frequência de deploy, os Desenvolvedores gastam <i>muito mais</i> tempo em tarefas valiosas. Destacam que sucesso depende de documentação, onboarding e cultura de plataforma-proprietário. Recomendam	Amostra pequena e não randomizada; foco em poucos contextos. Conclusões baseadas em dados limitados. Necessidade de validação externa.

				medir empiricamente e carga cognitiva (até com biofeedback).	
Allam (2022)	Explorar PE como serviço (PEaaS) via estudo de caso em nuvem.	Estudo de caso comparativo (autor afirma análise de empresas), revisão de abordagens de PE.	Casos de uso de PEaaS em empresas de cloud (não especifica quantas).	Relata que PEaaS melhora alinhamento dev/ops, reduz carga cognitiva.	Descrição geral, sem dados quantitativos claros. Amostra e método não detalhados. Limitações metodológicas não discutidas; resultado em certa medida promocional.
Oliveira <i>et al.</i> (2026)	Avaliar impacto de IDP na produtividade e DevEx em experimento controlado.	Estudo empírico (experimento comparativo controlado + pesquisa)	59 profissionais (survey) + 8 participantes (experimento laboratório AWS)	Observou que participantes usando a IDP completaram deploy mais rápido e menor dificuldade que o grupo manual. Confirmação empírica de benefícios.	Tamanho reduzido e não aleatório (auto-seleção). O experimento usa IDP própria ("feita para o estudo"), o que pode não refletir plataformas comerciais. Não mede impacto a longo prazo.
van de Kamp <i>et al.</i> (2024)	Propor modelo referência de Platform Engineering (PE-RM) para entendimento comum.	Trabalho conceitual (framework teórico)	– (base teórica em RM-ODP e bibliografia existente)	Introduz o <i>Platform Engineering Reference Model</i> (PE-RM) baseado no RM-ODP para padronizar vocabulário e estratégias. Fornece estrutura multi-vista para alinhar design de	Conceitual; não inclui validação empírica ou aplicação real em organizações. Utilidade e aplicabilidade prática precisam ser testadas.

				plataformas organizacionais e roadmap de implementação. Objetiva mitigar complexidade de adotar PE e acelerar resultados.	
--	--	--	--	---	--

Diagramas e imagens ilustrativas

A seguir, apresentamos diagramas para facilitar o entendimento das relações-chave:

- **Fluxo conceitual de Platform Engineering:** O diagrama abaixo sintetiza como engenharia de plataforma, via IDPs e automação, reduz carga cognitiva e melhora a experiência do desenvolvedor, resultando em maior produtividade e velocidade de entrega.

EP[Engenharia de Plataforma] --> IDP[IDP (Plataforma Interna)]
 IDP --> SS[Self-Service & Automação]
 SS --> CC[Redução de Carga Cognitiva]
 CC --> DevEx[Melhoria da DevEx]
 DevEx --> Prod[Mais Produtividade / Entregas rápidas]

- **Componentes de um Portal Interno do Desenvolvedor:** A figura abaixo ilustra exemplos de componentes e funcionalidades associados a um IDP, organizados em categorias como Developer Experience, Service Catalog, Observability e Governance. Fonte: elaborado pelos autores, com base em Anjum (2026).

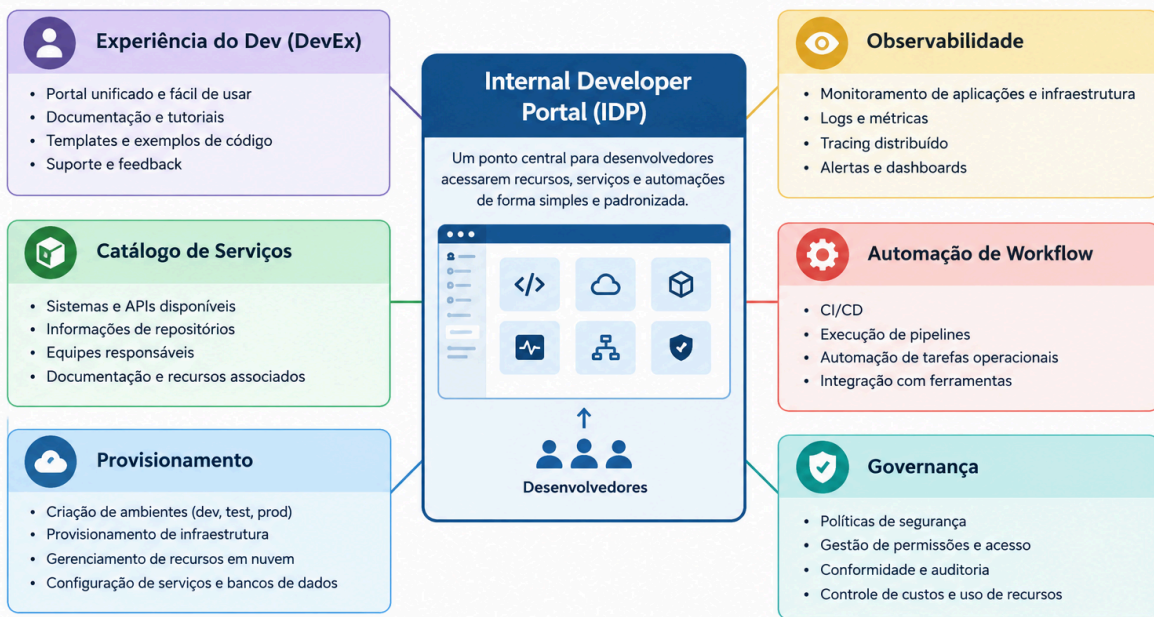


Figura: Exemplos de componentes e funcionalidades típicas em um portal interno de desenvolvedores (IDP), organizados em categorias como experiência do dev, catálogo de serviços, provisionamento, observabilidade, automação de workflow e governança. Adaptado de Anjum et al. (2026).

Conclusão

A engenharia de plataformas busca simplificar o ambiente de desenvolvimento e facilitar o acesso às ferramentas e recursos utilizados pelos desenvolvedores de uma empresa. Seu foco não está apenas na criação de uma nova plataforma, mas principalmente na melhoria da experiência de desenvolvimento. Recursos como catálogo de serviços, self-service, Golden-Paths e automações podem ajudar as organizações a reduzir tarefas repetitivas, melhorar a organização dos sistemas e tornar o desenvolvimento mais simples e padronizado. Ao mesmo tempo, a plataforma precisa ser planejada com cuidado para não aumentar ainda mais a complexidade. O objetivo principal deve ser facilitar o trabalho dos desenvolvedores e fornecer uma experiência mais organizada e eficiente.

Referências

- Guisao Baena, Y., Suescún Monsalve, E., Noreña Cardona, P. & Pardo, C. (2026). *Toward Platform Engineering, the Evolution of DevOps—A Systematic Mapping*. **Journal of Software: Evolution and Process**, 38(4), e70108. DOI: 10.1002/smr.70108.
- Pitkar, H. (2026). *Platform Engineering and Developer Experience: A Systematic Review of Concepts, Benefits and Future Directions*. **World Journal of Advanced Engineering Technology and Sciences**, 18(2), 241–248. DOI: 10.30574/wjaets.2026.18.2.0112.
- Anjum, M. M. A. (2026). *Platform Engineering and Internal Developer Portals: a Multivocal Literature Review*. **Frontiers in Computer Science**, 8:1814498. DOI: 10.3389/fcomp.2026.1814498.
- Seknametla, P. R. & Sunkara, R. (2023). *Platform Engineering and Internal Developer Platforms: Measuring Cognitive Load Reduction and Developer Productivity in Self-Service Infrastructure Models*. **International Journal of Computer Techniques**, 10(4), ISSN 2394-2231. (Disponível em ijctjournal.org).
- Allam, H. (2022). *Platform Engineering as a Service: Streamlining Developer Experience in Cloud Environments*. **Intl. Journal of Emerging Research in Engineering and Technology**, 9(3):50-59. DOI: 10.63282/3050-922X.IJERET-V3I3P106.
- Oliveira, L. et al. (2026). *O impacto da engenharia de plataforma na produtividade e experiência do desenvolvedor*. **Revista E&S – Engenharia e Sistemas**, n.p. (2026). Resumo disponível online.
- van de Kamp, R., Bakker, K. C. & Zhao, Z. (2024). *Paving the path towards platform engineering using a comprehensive reference model*. In: *Enterprise Design, Operations, and Computing (EDOC 2023)*. DOI: 10.1007/978-3-031-54712-6_11. (Resumido em Zenodo [48]).
- CIANCARINI, Paolo et al. *The Design and Realization of a Self-Hosted and Open-Source Agile Internal Development Platform*. **IEEE Access**, 2025, v. 13, p. 79516–79533. DOI: 10.1109/ACCESS.2025.3564141.
- ANJUM, Mateen Ali. **Platform engineering and internal developer portals: a multivocal literature review**. *Frontiers in Computer Science*, v. 8, 2026. DOI: 10.3389/fcomp.2026.1814498. Disponível em: [Frontiers in Computer Science](https://doi.org/10.3389/fcomp.2026.1814498)
- CIANCARINI, Paolo et al. **The design and realization of a self-hosted and open-source agile internal development platform**. *IEEE Access*, v. 13, p. 79516-79533, 2025. DOI: 10.1109/ACCESS.2025.3564141. Disponível em: [IEE Xplore](https://ieeexplore.ieee.org/document/9251141)
- GREILER, Michaela; STOREY, Margaret-Anne; NODA, Abi. **An actionable framework for understanding and improving developer experience**. *IEEE Transactions on Software Engineering*, v. 49, n. 4, p. 1411-1425, 2023. DOI: 10.1109/TSE.2022.3175660. Disponível em: [IEE Xplore](https://ieeexplore.ieee.org/document/9851141)
- NIEMINEN, Veikko. **Internal Developer Platform**. 2024. Dissertação (Mestrado) – Tampere University, Tampere, 2024. Disponível em: [Repositório da Tampere University](https://repository.tampere.fi/handle/10257/10257)
- Backstage Software Catalog. Instituição/projeto: Backstage. Ano: documentação consultada em 2026. Endereço: <https://backstage.io/docs/features/software-catalog/>. Justificativa:

documentação oficial para a parte de catálogo de software e organização de serviços. Backstage Software Templates. Instituição/projeto: Backstage. Ano: documentação consultada em 2026. Endereço: <https://backstage.io/docs/features/software-templates/>.

Notícia recente: “CNCF and SlashData Report Finds Platform Engineering Tools Maturing as Organizations Prepare for AI-Driven Infrastructure”. Instituição: Cloud Native Computing Foundation (CNCF). Data: 24 de março de 2026. Endereço: <https://www.cncf.io/announcements/2026/03/24/cncf-and-slashdata-report-finds-platform-engineering-tools-maturing-as-organizations-prepare-for-ai-driven-infrastructure/>. Justificativa: publicação institucional da CNCF sobre uma pesquisa com mais de 400 desenvolvedores profissionais e sobre a avaliação de ferramentas de platform engineering. A data está dentro dos 12 meses anteriores à pesquisa.

Vídeo de apoio: *Platform Engineering for Software Developers and Architects* — Daniel Bryant, Syntasso; CNCF, 2024.

Justificativa: vídeo técnico que apresenta conceitos de Platform Engineering, arquitetura de plataformas, impacto no ciclo de desenvolvimento de software e métricas relacionadas à experiência do desenvolvedor.

Bibliografia

Guisao Baena, Y., Suescún Monsalve, E., Noreña Cardona, P. & Pardo, C. (2026). *Toward Platform Engineering, the Evolution of DevOps—A Systematic Mapping*. **Journal of Software: Evolution and Process**, 38(4), e70108. DOI: 10.1002/smr.70108.

Pitkar, H. (2026). *Platform Engineering and Developer Experience: A Systematic Review of Concepts, Benefits and Future Directions*. **World Journal of Advanced Engineering Technology and Sciences**, 18(2), 241–248. DOI: 10.30574/wjaets.2026.18.2.0112.

Anjum, M. M. A. (2026). *Platform Engineering and Internal Developer Portals: a Multivocal Literature Review*. **Frontiers in Computer Science**, 8:1814498. DOI: 10.3389/fcomp.2026.1814498.

Seknametla, P. R. & Sunkara, R. (2023). *Platform Engineering and Internal Developer Platforms: Measuring Cognitive Load Reduction and Developer Productivity in Self-Service Infrastructure Models*. **International Journal of Computer Techniques**, 10(4), ISSN 2394-2231. (Disponível em ijctjournal.org).

Allam, H. (2022). *Platform Engineering as a Service: Streamlining Developer Experience in Cloud Environments*. **Intl. Journal of Emerging Research in Engineering and Technology**, 9(3):50-59. DOI: 10.63282/3050-922X.IJERET-V3I3P106.

Oliveira, L. et al. (2026). *O impacto da engenharia de plataforma na produtividade e experiência do desenvolvedor*. **Revista E&S – Engenharia e Sistemas**, n.p. (2026). Resumo disponível online.

van de Kamp, R., Bakker, K. C. & Zhao, Z. (2024). *Paving the path towards platform engineering using a comprehensive reference model*. In: *Enterprise Design, Operations, and Computing (EDOC 2023)*. DOI: 10.1007/978-3-031-54712-6_11. (Resumido em Zenodo [48]).

CIANCARINI, Paolo et al. *The Design and Realization of a Self-Hosted and Open-Source Agile Internal Development Platform*. **IEEE Access**, 2025, v. 13, p. 79516–79533. DOI: 10.1109/ACCESS.2025.3564141.

ANJUM, Mateen Ali. **Platform engineering and internal developer portals: a multivocal literature review**. *Frontiers in Computer Science*, v. 8, 2026. DOI: 10.3389/fcomp.2026.1814498. Disponível em: [Frontiers in Computer Science](#)

CIANCARINI, Paolo et al. **The design and realization of a self-hosted and open-source agile internal development platform**. *IEEE Access*, v. 13, p. 79516-79533, 2025. DOI: 10.1109/ACCESS.2025.3564141. Disponível em: [IEE Xplore](#)

GREILER, Michaela; STOREY, Margaret-Anne; NODA, Abi. **An actionable framework for understanding and improving developer experience**. *IEEE Transactions on Software Engineering*, v. 49, n. 4, p. 1411-1425, 2023. DOI: 10.1109/TSE.2022.3175660. Disponível em: [IEE Xplore](#)

NIEMINEN, Veikko. **Internal Developer Platform**. 2024. Dissertação (Mestrado) – Tampere University, Tampere, 2024. Disponível em: [Repositório da Tampere University](#)

Backstage Software Catalog. Instituição/projeto: Backstage. Ano: documentação consultada em 2026. Endereço: <https://backstage.io/docs/features/software-catalog/>. Justificativa: documentação oficial para a parte de catálogo de software e organização de serviços. Backstage Software Templates. Instituição/projeto: Backstage. Ano: documentação consultada em 2026. Endereço: <https://backstage.io/docs/features/software-templates/>.

Notícia recente: “CNCf and SlashData Report Finds Platform Engineering Tools Maturing as Organizations Prepare for AI-Driven Infrastructure”. Instituição: Cloud Native Computing Foundation (CNCf). Data: 24 de março de 2026. Endereço: <https://www.cncf.io/announcements/2026/03/24/cncf-and-slashdata-report-finds-platform-engineering-tools-maturing-as-organizations-prepare-for-ai-driven-infrastructure/>. Justificativa: publicação institucional da CNCf sobre uma pesquisa com mais de 400 desenvolvedores profissionais e sobre a avaliação de ferramentas de platform engineering. A data está dentro dos 12 meses anteriores à pesquisa.

Vídeo de apoio: *Platform Engineering for Software Developers and Architects* — Daniel Bryant, Syntasso; CNCf, 2024.

Justificativa: vídeo técnico que apresenta conceitos de Platform Engineering, arquitetura de plataformas, impacto no ciclo de desenvolvimento de software e métricas relacionadas à experiência do desenvolvedor.

Oposição – Engenharia de Dados

A engenharia de dados ocupa uma posição central em sistemas analíticos, plataformas digitais e aplicações de inteligência artificial, mas seus benefícios não surgem automaticamente da adoção de nuvem, ferramentas de ETL ou arquiteturas modernas. A literatura científica mostra que a utilidade desses sistemas depende da qualidade dos dados, da confiabilidade dos pipelines, da capacidade de adaptação a mudanças e da coordenação entre equipes técnicas e áreas de negócio. Quando esses elementos são negligenciados, erros podem atravessar várias etapas, permanecer invisíveis e afetar relatórios, decisões e modelos.

Esta pesquisa apresenta uma contraposição crítica ao discurso excessivamente otimista sobre a área. O objetivo não é negar a importância da engenharia de dados, mas demonstrar que sua implantação envolve limitações técnicas, organizacionais, econômicas e éticas. As evidências analisadas indicam que pipelines são sistemas sociotécnicos: sua confiabilidade depende de código e infraestrutura, mas também de documentação, responsabilidades, conhecimento do domínio, governança e avaliação contínua.

Problemas Técnicos

Baixa qualidade de dados: dados ausentes, tipos incorretos, duplicidades, erros de preenchimento, formatos incompatíveis e falta de atualização podem comprometer todo o fluxo de processamento. A qualidade não é uma propriedade única, mas um conjunto de dimensões como completude, precisão, consistência, atualidade, unicidade e representatividade. Por isso, uma base pode ser tecnicamente válida e ainda assim ser inadequada para determinado uso.

Foidl et al. (2024) desenvolveram uma taxonomia com 41 fatores que influenciam a capacidade de pipelines entregarem dados de qualidade. O trabalho combinou revisão multivocal da literatura, entrevistas com oito especialistas e mineração de projetos do GitHub e discussões do Stack Overflow. Entre os problemas analisados, tipos incorretos de dados representaram 33% das causas, enquanto 35% apareceram durante a etapa de limpeza. Questões de integração e ingestão corresponderam a 47% das dúvidas examinadas. Embora a amostra não represente todas as organizações, os resultados mostram que os problemas se concentram tanto nos dados quanto nas operações realizadas sobre eles.

Mohammed et al. (2025) avaliaram seis dimensões de qualidade em 19 algoritmos, envolvendo tarefas de classificação, regressão e agrupamento em dez conjuntos de dados. Valores ausentes, baixa precisão das características e erros nas variáveis-alvo produziram impactos importantes em diferentes cenários. Entretanto, duplicações e inconsistências de representação tiveram efeitos menores em alguns algoritmos e bases. Isso indica que nem todo defeito possui a mesma gravidade e que a correção deve ser priorizada conforme o uso, o risco e o custo.

Qualidade do próprio pipeline: não basta que os dados de entrada sejam adequados. O pipeline precisa executar corretamente suas transformações, preservar regras de negócio, registrar dependências e entregar o resultado esperado. Um fluxo pode terminar sem erro técnico e ainda produzir informações incompletas ou semanticamente incorretas. Assim, disponibilidade do serviço não equivale a confiabilidade dos dados.

Testes tradicionais costumam verificar execução, latência, utilização de recursos ou presença de colunas. Essas verificações são necessárias, mas não comprovam que o significado dos dados permaneceu igual. Uma variável pode conservar o mesmo nome e o mesmo tipo enquanto passa a representar outra regra de negócio. Mudanças desse tipo podem permanecer invisíveis até que um indicador, relatório ou modelo produza uma decisão inadequada.

Complexidade e integração de sistemas: pipelines frequentemente conectam sistemas legados, bancos relacionais e não relacionais, APIs, arquivos, eventos e fluxos em tempo real. Cada fonte pode adotar formatos, frequências e conceitos distintos. A integração exige não apenas compatibilidade técnica, mas também alinhamento semântico entre as equipes que produzem e consomem os dados.

Munappy, Bosch e Olsson (2020) analisaram cinco casos de uso em três empresas e identificaram dificuldades relacionadas à entrada constante de novas fontes, incompatibilidade de formatos, escalabilidade, infraestrutura e organização. O aumento de componentes e conectores tornava o pipeline mais difícil de compreender e manter. O estudo também encontrou benefícios na automação, como rastreabilidade, tolerância a falhas e redução de erros humanos, demonstrando que os resultados dependem da maturidade da implementação.

Escalabilidade e desempenho: o crescimento do volume, da velocidade e da variedade dos dados amplia a demanda por armazenamento, processamento, transferência e monitoramento. Pipelines legados podem apresentar gargalos, perda de dados ou aumento de custos quando novas fontes e consumidores são adicionados. A escalabilidade, portanto, não consiste apenas em processar mais dados, mas em fazê-lo sem perder controle, qualidade e previsibilidade.

A adoção de processamento distribuído e serviços em nuvem amplia a capacidade disponível, mas também introduz dependência de configuração, custos variáveis e complexidade operacional. Uma arquitetura tecnicamente escalável pode continuar inadequada se gerar duplicações, reprocessamentos desnecessários ou dificuldade para localizar a origem de um erro.

Data drift e concept drift: dados de produção mudam ao longo do tempo em razão de alterações no comportamento de usuários, sensores, processos empresariais, sistemas de origem ou condições externas. Lu et al. (2019) explicam que o tratamento de concept drift envolve detectar a mudança, compreender sua natureza e adaptar o sistema. A simples presença de monitoramento não resolve o problema, pois diferentes mudanças exigem respostas diferentes.

Dong et al. (2024) demonstram que nem todo data drift prejudica o desempenho. Retreinar modelos ou reprocessar grandes volumes sempre que uma alteração é detectada pode elevar os custos sem gerar benefícios. Os autores propõem identificar regiões da distribuição em que a mudança está relacionada à baixa precisão antes de decidir pela

intervenção. Esse resultado mostra que a automação excessivamente sensível também pode causar desperdício e fadiga operacional.

Manutenção, evolutividade e dívida técnica: alterações de esquema, requisitos, regras de negócio e dependências podem quebrar pipelines ou modificar silenciosamente seus resultados. Sculley et al. (2015) mostram que sistemas dependentes de dados acumulam dívida técnica por meio de consumidores não declarados, dependências ocultas, configurações complexas e ciclos de retroalimentação.

Uma tabela criada para uma finalidade pode passar a alimentar relatórios, aplicações e modelos sem que a equipe responsável conheça todos os consumidores. Quando sua estrutura ou semântica muda, o impacto se espalha pelo ecossistema. A dívida cresce quando faltam documentação, versionamento, linhagem e processos para comunicar mudanças. Dessa forma, a manutenção inclui não apenas corrigir código, mas compreender relações entre fontes, transformações e usos posteriores.

Problemas Organizacionais

Desconexão estratégico-operacional: equipes técnicas podem otimizar tempo de execução, disponibilidade e volume processado sem demonstrar como esses resultados apoiam decisões ou objetivos institucionais. A ausência de critérios de valor permite que plataformas complexas continuem crescendo mesmo quando seus consumidores, responsabilidades e benefícios não estão claramente definidos.

Heck (2024) realizou um estudo de mapeamento sobre engenharia de dados para sistemas de IA e encontrou 25 trabalhos publicados entre 2019 e junho de 2023. Os estudos apresentavam terminologias, escopos e níveis de validação distintos. Muitos se concentravam em pipelines de projetos individuais, enquanto poucos abordavam arquiteturas organizacionais completas. Essa fragmentação dificulta a criação de práticas universais e mostra que soluções avaliadas em um contexto não podem ser automaticamente generalizadas.

Responsabilidade distribuída e cascatas de dados: qualidade depende de produtores, engenheiros, analistas, especialistas do domínio, segurança, privacidade, gestão e consumidores. Quando essas responsabilidades não são explicitadas, problemas locais podem se acumular e aparecer apenas nas etapas finais. Sambasivan et al. (2021) denominam esse fenômeno de cascatas de dados.

O estudo entrevistou 53 profissionais de aplicações de IA de alto impacto em diferentes países. Dentro dessa amostra, 92% relataram pelo menos uma cascata. Entre as causas estavam documentação insuficiente, diferenças entre os ambientes de coleta e produção, ausência de conhecimento do domínio, conflitos de prioridade e baixa valorização do trabalho de coleta e curadoria. As consequências incluíam retrabalho, previsões incorretas e perda de confiança. Como o estudo se concentra em IA de alto impacto, seus resultados não representam todos os projetos, mas demonstram empiricamente que problemas de dados também possuem origem social e organizacional.

Escassez e diversidade de habilidades: engenharia de dados exige conhecimentos de bancos, programação, sistemas distribuídos, nuvem, segurança, qualidade, governança e domínio de negócio. A dificuldade não está somente em contratar especialistas, mas em coordenar profissionais com formações e prioridades diferentes. Quando os dados são tratados como responsabilidade exclusiva da equipe técnica, decisões sobre significado, representatividade e uso podem ser tomadas sem participação de especialistas do domínio.

Governança e custos: catálogos, linhagem, contratos, monitoramento e políticas de acesso exigem investimento e manutenção. Sem governança, podem surgir bases redundantes, métricas incompatíveis e fontes sem responsável. Entretanto, adicionar ferramentas não resolve automaticamente esses problemas. Um catálogo desatualizado ou um contrato abandonado pode criar uma falsa sensação de controle.

Retorno sobre investimento: a literatura analisada não sustenta percentuais universais sobre receitas perdidas nem permite afirmar que a maioria dos projetos de engenharia de dados fracassa financeiramente. O problema cientificamente defensável é a falta de métricas padronizadas que conectem práticas técnicas específicas aos resultados organizacionais. Disponibilidade, latência e volume medem a operação, mas não demonstram isoladamente geração de valor.

A avaliação deve considerar cada caso de uso, incluindo custo total, tempo de recuperação, quantidade de incidentes, retrabalho, decisões apoiadas e risco evitado. Um pipeline caro pode ser justificável em uma aplicação crítica, enquanto um fluxo de baixo custo pode ser desperdício se ninguém utiliza seus resultados.

Aspectos Éticos e de Governança

Viés e representatividade: decisões tomadas durante coleta, seleção, limpeza, integração e exclusão de registros podem alterar a representação de grupos antes mesmo da etapa de análise. Dados históricos podem incorporar desigualdades, e variáveis aparentemente neutras podem funcionar como aproximações de características sensíveis. Se esses problemas não forem documentados e avaliados, os sistemas posteriores podem reproduzi-los.

Priestley, O'Donnell e Simperl (2023) mostram que requisitos de qualidade relevantes para pipelines de aprendizado de máquina envolvem diferentes participantes e não se limitam à precisão técnica. Sambasivan et al. (2021) também demonstram que falta de documentação, conhecimento do domínio e incentivos adequados pode produzir efeitos negativos acumulados. A engenharia de dados participa desse processo porque define quais registros entram, como são transformados e quais informações permanecem disponíveis.

Privacidade e segurança: pipelines processam dados pessoais em múltiplas etapas, cópias e ambientes. Quanto maior a quantidade de integrações, maior a superfície de exposição. Controle de acesso, minimização, retenção, criptografia e rastreabilidade precisam ser considerados desde o desenho do fluxo. A anonimização também não deve ser tratada como solução automática, pois sua eficácia depende do contexto e da possibilidade de combinação com outras fontes.

Auditabilidade: decisões baseadas em dados exigem capacidade de reconstruir a origem, as transformações e as versões utilizadas. Sem linhagem, é difícil identificar quando um erro foi introduzido, quem foi afetado e quais resultados precisam ser corrigidos. A auditabilidade deve incluir registros técnicos, responsáveis e justificativas para alterações de regras.

Evidências Empíricas e Estudos de Caso

Qualidade de pipelines em projetos públicos: Foidl et al. (2024) identificaram 41 fatores de influência, tipos incorretos como causa recorrente e concentração de problemas nas etapas de limpeza, integração e ingestão. A combinação de revisão, entrevistas e mineração de repositórios fortalece a análise, mas projetos públicos e fóruns técnicos não representam todas as empresas.

Gestão de pipelines em empresas: Munappy et al. (2020) observaram cinco casos de uso em três organizações. Os autores encontraram problemas de infraestrutura, qualidade e coordenação, mas também ganhos de rastreabilidade e redução de erros. A principal limitação é o número pequeno de empresas, que impede generalizações amplas.

Cascatas de dados em aplicações críticas: Sambasivan et al. (2021) registraram cascatas em entrevistas com 53 profissionais. A pesquisa mostra como dificuldades aparentemente pequenas podem se acumular e permanecer invisíveis. Por se basear em relatos e aplicações de IA de alto impacto, deve ser usada como evidência contextual, não como taxa universal para toda engenharia de dados.

Qualidade e desempenho de algoritmos: Mohammed et al. (2025) avaliaram 19 algoritmos e mostraram que diferentes defeitos produzem impactos distintos. O desenho controlado permite comparações, mas não reproduz todas as dependências de produção. O estudo sustenta a necessidade de priorização, não a ideia de que qualquer imperfeição deve ser eliminada a qualquer custo.

Drift em pipelines de aprendizado de máquina: Dong et al. (2024) demonstraram que algumas mudanças de distribuição são prejudiciais e outras não. A intervenção deve considerar o impacto sobre o desempenho, evitando respostas automáticas indiscriminadas. A evidência se aplica diretamente a pipelines de ML e deve ser generalizada com cautela para outros sistemas.

Limitações das Soluções Propostas

Observabilidade: métricas de atualidade, volume, esquema, distribuição e linhagem melhoram a detecção de anomalias. Entretanto, só é possível alertar sobre propriedades conhecidas ou comportamentos que se desviam de um padrão. Uma mudança de significado pode permanecer invisível se não alterar a estrutura ou a distribuição de maneira detectável.

Validação automatizada: Schelter et al. (2018) apresentaram um sistema de verificação em larga escala baseado em restrições declarativas e detecção de anomalias. Breck et al. (2019) descreveram um sistema integrado ao TFX e utilizado por centenas de equipes para monitorar grandes volumes de dados de produção. Esses trabalhos mostram benefícios na detecção antecipada e na redução do tempo de depuração.

Apesar disso, a validação depende de expectativas previamente definidas. Valores ausentes, mudanças de tipo e alterações de volume são mais fáceis de verificar que transformações semânticas, novos vieses ou problemas desconhecidos. Por isso, ferramentas automáticas devem complementar conhecimento do domínio e revisão humana.

Contratos, catálogos e linhagem: contratos podem definir esquemas, semântica, responsabilidades e níveis de serviço; catálogos facilitam a descoberta; e linhagem registra origens e transformações. Contudo, todos exigem atualização. Se os metadados não acompanham a realidade, tornam-se novas fontes de inconsistência.

Pipelines autoadaptativos: mecanismos de autocorreção são promissores para alterações de esquema e drift, mas muitas propostas ainda são avaliadas em protótipos ou experimentos controlados. Uma correção equivocada pode ampliar uma falha localizada. Mudanças com impacto elevado precisam de limites de confiança, validação, reversão e supervisão humana.

Recomendações Práticas e Pesquisa Futura

Testes em diferentes níveis: além da execução do pipeline, devem ser testados esquema, tipos, completude, unicidade, volume, distribuição, atualidade e regras do domínio. Resultados críticos precisam de validação semântica e comparação com expectativas de negócio.

Priorização baseada em impacto: problemas de qualidade devem ser classificados conforme consumidores, risco e decisões afetadas. Dados utilizados em aplicações críticas exigem controles mais rigorosos que bases exploratórias de baixo risco.

Monitoramento responsável de drift: mudanças devem ser acompanhadas, mas não precisam acionar correções indiscriminadamente. A resposta deve considerar se houve degradação real e qual componente foi afetado.

Registro de consumidores e versionamento: tabelas, eventos, relatórios e modelos precisam ter responsáveis e consumidores identificados. Alterações de esquema ou semântica devem ser versionadas e comunicadas antes da implantação.

Supervisão humana e reversão: correções automáticas devem possuir limites de confiança, validação posterior e mecanismo de rollback. Mudanças de alto impacto precisam ser revisadas por profissionais técnicos e especialistas do domínio.

Governança compartilhada: qualidade não deve ser responsabilidade exclusiva do engenheiro de dados. Produtores, especialistas do domínio, segurança, privacidade, gestão e consumidores precisam participar da definição de regras e critérios de aceitação.

Métricas associadas ao uso: além de disponibilidade e latência, as organizações podem medir incidentes, retrabalho, tempo de recuperação, decisões afetadas e satisfação dos consumidores. Essas métricas aproximam a operação técnica do valor esperado.

Síntese dos Problemas e Recomendações

Problema ou categoria	Evidências e implicações	Recomendações
Qualidade de dados	Valores ausentes, tipos incorretos, baixa precisão e falta de representatividade podem comprometer análises e modelos. O impacto varia conforme o uso.	Aplicar métricas de completude, precisão, consistência, atualidade e representatividade, priorizadas pelo risco.
Qualidade do pipeline	O fluxo pode terminar sem falha e ainda entregar dados semanticamente incorretos.	Testar transformações e regras do domínio, não apenas infraestrutura e execução.
Integração	Fontes heterogêneas e novos conectores ampliam incompatibilidades e complexidade.	Padronizar interfaces, documentar semântica e validar novas fontes antes da produção.
Drift	Mudanças de distribuição ou conceito podem degradar resultados, mas nem todo drift é prejudicial.	Avaliar impacto antes de retreinar, reprocessar ou alterar componentes.
Dívida de dados	Dependências ocultas e consumidores não declarados tornam mudanças caras e arriscadas.	Manter versionamento, linhagem, responsáveis e registro de consumidores.
Escalabilidade e custos	Mais volume e componentes podem gerar gargalos, duplicação e custos imprevisíveis.	Monitorar custo por caso de uso, eliminar reprocessamentos e revisar capacidade periodicamente.
Coordenação	Responsabilidades ambíguas e falta de conhecimento do domínio produzem cascatas de dados.	Definir ownership compartilhado e envolver produtores, consumidores e especialistas.
Observabilidade	Métricas detectam violações conhecidas, mas podem ignorar mudanças semânticas.	Combinar observabilidade, validação de domínio e revisão humana.
Ética e privacidade	Coleta e transformação podem propagar vieses ou expor dados pessoais.	Adotar minimização, controle de acesso, auditoria, rastreabilidade e avaliação de representatividade.
Retorno sobre investimento	Métricas técnicas não demonstram isoladamente valor financeiro ou decisório.	Relacionar desempenho técnico a incidentes, risco, uso, produtividade e resultados organizacionais.

Lacunas de Pesquisa e Sugestões de Melhorias

A literatura ainda apresenta poucas comparações de longo prazo entre arquiteturas de dados. Muitos trabalhos propõem frameworks ou demonstram protótipos, mas não acompanham sua manutenção durante anos em ambientes de produção. Também faltam

métricas padronizadas para qualidade de pipelines, dívida de dados, produtividade e retorno sobre investimento.

Outra lacuna é a validação de sistemas autoadaptativos em ambientes reais. Testes com falhas ou drifts inseridos artificialmente são úteis para comparar métodos, porém não reproduzem alterações simultâneas, regras de negócio, dependências desconhecidas e limitações humanas. Estudos futuros devem incluir implantações longitudinais, diferentes setores e indicadores de custo, segurança e qualidade.

Também são necessárias pesquisas específicas sobre ética na engenharia de dados. Grande parte da literatura concentra o debate nos modelos de IA, embora decisões tomadas na coleta, limpeza, integração e exclusão de registros possam alterar a representação de grupos antes do treinamento. A pesquisa futura deve relacionar qualidade, privacidade, justiça e governança ao longo de todo o ciclo de dados.

Por fim, DataOps, contratos, catálogos, observabilidade e arquiteturas autoadaptativas precisam de avaliações comparáveis. O fato de uma ferramenta detectar anomalias em um experimento não demonstra que ela reduz incidentes, custos ou retrabalho em organizações de diferentes portes. A área necessita de replicações, benchmarks públicos e critérios claros de sucesso.

Conclusão

As evidências não demonstram que a engenharia de dados seja dispensável. Ao contrário, mostram que ela é necessária porque dados, fontes, consumidores e requisitos mudam continuamente. A contraposição central é que seus benefícios não aparecem de forma automática com a adoção de nuvem, plataformas modernas ou maior volume de processamento.

Pipelines exigem manutenção contínua; erros podem atravessar várias etapas; dependências podem permanecer ocultas; e soluções automáticas possuem limites. A confiabilidade depende da combinação entre tecnologia, processos, conhecimento do domínio, supervisão e governança. Uma infraestrutura não deve ser considerada bem-sucedida apenas porque processa dados rapidamente, mas porque preserva significado, qualidade, rastreabilidade, responsabilidade e utilidade ao longo do tempo.

Referências

- BRECK, E.; ZINKEVICH, M.; POLYZOTIS, N.; WHANG, S.; ROY, S. Data Validation for Machine Learning. *Proceedings of SysML*, 2019.
- DONG, S.; WANG, Q.; SAHRI, S.; PALPANAS, T.; SRIVASTAVA, D. Efficiently Mitigating the Impact of Data Drift on Machine Learning Pipelines. *Proceedings of the VLDB Endowment*, v. 17, n. 11, p. 3072-3081, 2024. DOI: 10.14778/3681954.3681984.
- FOIDL, H.; GOLENDUKHINA, V.; RAMLER, R.; FELDERER, M. Data Pipeline Quality: Influencing Factors, Root Causes of Data-related Issues, and Processing Problem Areas for Developers. *Journal of Systems and Software*, v. 207, 111855, 2024. DOI: 10.1016/j.jss.2023.111855.
- HECK, P. What About the Data? A Mapping Study on Data Engineering for AI Systems. In: *Conference on AI Engineering - Software Engineering for AI*, CAIN 2024. ACM, 2024. DOI: 10.1145/3644815.3644954.
- KLETTKE, M.; STÖRL, U. Four Generations in Data Engineering for Data Science. *Datenbank-Spektrum*, v. 22, p. 59-66, 2022. DOI: 10.1007/s13222-021-00399-3.

- LU, J.; LIU, A.; DONG, F.; GU, F.; GAMA, J.; ZHANG, G. Learning under Concept Drift: A Review. *IEEE Transactions on Knowledge and Data Engineering*, v. 31, n. 12, p. 2346-2363, 2019. DOI: 10.1109/TKDE.2018.2876857.
- MOHAMMED, S.; BUDACH, L.; FEUERPFEL, M.; IHDE, N.; NATHANSEN, A.; NOACK, N.; PATZLAFF, H.; NAUMANN, F.; HARMOUCH, H. The Effects of Data Quality on Machine Learning Performance on Tabular Data. *Information Systems*, v. 132, 102549, 2025. DOI: 10.1016/j.is.2025.102549.
- MUNAPPY, A. R.; BOSCH, J.; OLSSON, H. H. Data Pipeline Management in Practice: Challenges and Opportunities. In: *Product-Focused Software Process Improvement, PROFES 2020*. p. 168-184, 2020. DOI: 10.1007/978-3-030-64148-1_11.
- PRIESTLEY, M. A.; O'DONNELL, F.; SIMPERL, E. A Survey of Data Quality Requirements That Matter in ML Development Pipelines. *Journal of Data and Information Quality*, v. 15, n. 2, 2023. DOI: 10.1145/3592616.
- SAMBASIVAN, N.; KAPANIA, S.; HIGHFILL, H.; AKRONG, D.; PARITOSH, P. K.; AROYO, L. M. Everyone Wants to Do the Model Work, Not the Data Work: Data Cascades in High-Stakes AI. In: *CHI Conference on Human Factors in Computing Systems*, 2021. DOI: 10.1145/3411764.3445518.
- SCHELTER, S.; LANGE, D.; SCHMIDT, P.; CELIKEL, M.; BIESSMANN, F.; GRAFBERGER, A. Automating Large-Scale Data Quality Verification. *Proceedings of the VLDB Endowment*, v. 11, n. 12, p. 1781-1794, 2018. DOI: 10.14778/3229863.3229867.
- SCULLEY, D.; HOLT, G.; GOLOVIN, D.; DAVYDOV, E.; PHILLIPS, T.; EBNER, D.; CHAUDHARY, V.; YOUNG, M.; CRESPO, J.; DENNISON, D. Hidden Technical Debt in Machine Learning Systems. *Advances in Neural Information Processing Systems*, v. 28, 2015.
- TEBERNUM, D.; ALTENDEITERING, M.; HOWAR, F. DERM: A Reference Model for Data Engineering. In: *International Conference on Data Science, Technology and Applications, DATA 2021*. p. 165-175, 2021. DOI: 10.5220/0010517301650175.

Complementação

1. Resumo da apresentação anterior:

Introdução do problema e da engenharia de plataformas: Vimos o problema inicial que introduziu a necessidade da engenharia de plataformas e os fundamentos utilizados nela.

Expansão na metodologia: Foi explicado em mais profundidade os conceitos como golden paths que criam guias para usuários, Self-service que permite desenvolvimento com menos dependências e DevEx, que oferece feedback e comunicação da experiência do desenvolvedor durante desenvolvimento e o uso da plataforma.

Visão Prática: Exploramos esses conceitos na prática por meio de um produto minimamente viável, em qual foi visto a interface que um usuário dessa plataforma iria utilizar, as informações relevantes da experiência do desenvolvedor, e o streamlining oferecido por Self-service no processo de criação.

Limitações: Discutimos as limitações e os pitfalls de engenharia de plataforma, como aumento de complexidade e carga cognitiva por meio de criar ainda mais abstração entre o usuário e o código, terminologia e documentação escassa, e a necessidade da aplicação não restrigente das ferramentas de engenharia de plataformas.

Possível Futuro: Teve Teorias do futuro da plataforma, como a utilização de AI para adicionar flexibilidade e adaptação, o paradigm shift da plataforma ser mais reconhecida como um produto, e o amadurecimento de frameworks e dados mais concretos.

Conclusão: Foi concluído que engenharia de plataformas aplicada efetivamente utilizar feedback e organização para adaptar a plataforma ao invés de adicionar mais ferramentas sem planejamento, tudo a fim de diminuir a carga cognitiva do desenvolvedor junto com automatizar e padronizar o desenvolvimento dentro da plataforma.

2. Artigo científico

Platform Engineering and Internal Developer Portals: a Multivocal Literature Review – Anjum (2026)

O artigo de Mateen Ali Anjum, publicado em maio de 2026 na *Frontiers in Computer Science*, realiza uma revisão multivocal sobre Engenharia de Plataformas e Internal Developer Portals. Diferente de uma revisão que utiliza apenas artigos científicos, o estudo também considera literatura produzida pela indústria, como relatórios técnicos e pesquisas de mercado. Isso acontece porque Platform Engineering ainda é uma área relativamente recente e grande parte das práticas utilizadas atualmente surgiu primeiro dentro das empresas e comunidades técnicas.

O estudo começou com 143 fontes candidatas e, após critérios de seleção e avaliação de qualidade, chegou a 88 fontes utilizadas na análise final. Destas, 44 foram classificadas como fontes acadêmicas de maior qualidade, 26 incluíram estudos acadêmicos de menor nível, teses e preprints, e 18 eram materiais da chamada literatura cinzenta. O autor separou as fontes dessa maneira justamente para não tratar uma pesquisa acadêmica e uma publicação de fornecedor como evidências de mesmo peso.

Esse ponto é importante para o debate porque o outro grupo criticou corretamente a forte influência da indústria sobre o tema. O próprio artigo confirma que existe uma situação diferente do que normalmente acontece em outras áreas da Engenharia de Software: em Platform Engineering, a prática avançou antes da pesquisa acadêmica. O estudo identifica aproximadamente três anos de diferença entre a consolidação do conceito dentro da indústria e os primeiros trabalhos acadêmicos focados diretamente em Engenharia de Plataformas.

Portanto, o artigo não tenta apresentar Engenharia de Plataformas como uma solução comprovada para todos os casos. Pelo contrário, ele ajuda a separar aquilo que já possui evidência mais consistente daquilo que ainda é principalmente uma prática promovida pela indústria.

Principais componentes identificados

Ao comparar as fontes, Anjum identifica seis grupos de capacidades que aparecem repetidamente nas plataformas internas:

Developer Experience, catálogo de serviços, capacidades da plataforma, observabilidade, automação de workflows e governança/padronização.

Isso também ajuda a mostrar que uma IDP não deve ser entendida apenas como um portal ou uma interface gráfica. O portal é apenas uma maneira de disponibilizar capacidades. A Engenharia de Plataformas envolve um conjunto maior

de elementos que tentam reduzir o esforço necessário para que desenvolvedores encontrem informações, provisionam recursos, criem serviços ou utilizem infraestrutura.

O objetivo central continua sendo reduzir a complexidade enfrentada pelas equipes de produto sem necessariamente remover toda a complexidade existente no sistema.

Plataforma como produto

Um dos pontos mais importantes do artigo é o conceito de Platform as a Product.

Nesse modelo, a plataforma é tratada como um produto interno e os desenvolvedores são tratados como seus usuários. A equipe de plataforma não deveria simplesmente decidir quais ferramentas as outras equipes devem utilizar.

Ela precisa compreender os fluxos de trabalho existentes, descobrir quais problemas realmente precisam ser resolvidos e acompanhar se as capacidades entregues estão sendo utilizadas.

O artigo relaciona isso ao princípio da “thinnest viable platform”, ou plataforma mínima viável: a plataforma deve fazer somente aquilo que realmente precisa fazer, evitando criar abstrações e funcionalidades desnecessárias. Esse ponto responde diretamente à crítica de over-engineering.

Uma plataforma que tenta resolver todos os problemas possíveis pode acabar se transformando em uma nova fonte de complexidade. O próprio Anjum identifica a abstração prematura como um padrão de falha: equipes constroem Golden Paths e automações sofisticadas antes mesmo de entender quais problemas dos desenvolvedores justificam esse investimento.

Nesse sentido, o artigo não contradiz a existência do risco apresentado pelo outro grupo. Ele explica como esse risco deve ser evitado.

Golden Paths e autonomia

Outro ponto importante é a relação entre padronização e autonomia.

Os Golden Paths procuram fornecer um caminho pronto para tarefas que acontecem com frequência. Um novo micro serviço, por exemplo, pode receber automaticamente uma estrutura de projeto, pipeline, políticas básicas de segurança, observabilidade e configurações necessárias.

Porém, a literatura analisada por Anjum recomenda que esses caminhos funcionem como facilitadores e não como mandatos. O próprio estudo conclui que

plataformas devem preservar a autonomia do desenvolvedor e que Golden Paths rígidos podem gerar resistência.

Essa distinção é importante porque o outro grupo apresentou justamente o risco da chamada Golden Cage.

A crítica faz sentido quando uma organização transforma uma recomendação em obrigação. Porém, nesse caso, o problema não é a existência do Golden Path. O problema está na maneira como ele foi utilizado.

Um Golden Path bem aplicado procura fazer o caminho mais comum ser também o mais fácil, sem impedir situações que realmente precisam de uma solução diferente.

Como avaliar se a plataforma funciona

Outro ponto muito relevante do artigo é que ele critica a ideia de avaliar Engenharia de Plataformas utilizando apenas uma métrica.

Anjum separa três grupos principais de indicadores:

DORA, relacionado ao desempenho de entrega, incluindo frequência de deploy, lead time, taxa de falha e tempo de recuperação, SPACE/DevEx, relacionado à experiência do desenvolvedor, satisfação, fluxo de trabalho, eficiência e comunicação e métricas específicas de plataforma, como adoção de self-service, tempo de onboarding, utilização dos Golden Paths e satisfação dos usuários internos.

Um problema identificado pelo artigo é que métricas específicas de Engenharia de Plataformas ainda possuem pouca validação acadêmica.

Isso significa que uma empresa não deveria concluir que sua plataforma funciona apenas porque aumentou a frequência de deploy, por exemplo. É possível aumentar uma métrica técnica enquanto a experiência dos desenvolvedores piora.

O artigo defende, portanto, uma combinação entre métricas técnicas e feedback dos próprios desenvolvedores.

Limitações e falta de evidências

Talvez a parte mais importante para o debate seja que Anjum não esconde as limitações da área.

O estudo mostra que afirmações mais gerais, como a existência de relação entre qualidade das ferramentas e produtividade, possuem um conjunto razoável de evidências. Porém, quanto mais específica é a afirmação sobre uma funcionalidade de Platform Engineering, menor tende a ser a quantidade de estudos disponíveis.

Por exemplo, o artigo encontrou alguma evidência para melhorias na descoberta de serviços e onboarding, mas identifica práticas amplamente utilizadas comercialmente, como determinados scorecards de IDPs, sem evidência empírica revisada por pares demonstrando sua efetividade. Isso é importante porque permite uma posição mais equilibrada:

Engenharia de Plataformas não deve ser apresentada como uma solução cientificamente comprovada em qualquer contexto.

Porém, a ausência de validação para determinadas práticas também não comprova que a abordagem é ineficaz. Ela demonstra que existe uma área ainda em amadurecimento e que os resultados precisam ser avaliados dentro das próprias organizações.

Importância do artigo para nossa pesquisa

O principal valor desse artigo para a complementação é justamente permitir responder às críticas sem precisar negá-las.

Ele reconhece praticamente todos os problemas importantes apresentados pelo outro grupo: over-engineering, risco de perda de autonomia, dificuldade de medir resultados, maturidade organizacional e influência da indústria.

Porém, o estudo apresenta esses problemas como condições que precisam ser consideradas durante a implantação, e não como provas de que Engenharia de Plataformas é uma abordagem inviável.

A principal interpretação que podemos retirar do artigo é que uma boa iniciativa de Platform Engineering depende mais de resolver problemas reais dos desenvolvedores do que da quantidade de funcionalidades existentes na plataforma.

3. Notícia recente

Em um portal especializado em tecnologia, denominado InfoQ, que é voltado para profissionais da área de TI, publicou recentemente que um grupo está alcançando conformidade como um time de engenharia de plataforma, ajudando os desenvolvedores.

Essa equipe buscou implementar práticas de governança e conformidade, mas obteve resistência dos desenvolvedores devido aos novos processos impostos, por conta da complexidade dos fluxos de trabalho e documentações inadequadas.

Davide De Paolis, um profissional da tecnologia, relatou que após a criação da equipe de plataforma, foram planejadas iniciativas como um catálogo de serviços, uma plataforma interna para desenvolvedores e um centro de excelência em nuvem.

Impactos relacionados

Entretanto, a adoção dessas mudanças gerou dificuldades como: desenvolvedores tendo que lidar com várias contas AWS, novas permissões, configurações adicionais e conceitos desconhecidos. Além disso, a documentação era extensa, desatualizada e difícil de consultar, prejudicando a experiência dos desenvolvedores e aumentando as solicitações de suporte.

Um exemplo foi a implementação de etiquetas (tags) nos recursos da AWS, utilizadas para identificar responsáveis, controlar custos e facilitar a governança. Uma primeira tentativa fracassou porque as regras eram vistas como burocracia. Na segunda tentativa, a equipe simplificou as exigências, definiu um conjunto mínimo de tags e utilizou ferramentas da AWS para prevenir e detectar recursos fora do padrão.

Tendências relacionadas

A tendência é seguir uma estratégia de três etapas: prevenção: estabelecer regras e padrões para evitar problemas desde a criação dos recursos. Detecção: identificar recursos que não atendem às exigências. Comunicação: informar as equipes sobre os problemas e explicar a importância das mudanças antes de aplicar restrições mais rigorosas.

A governança deve priorizar os riscos mais importantes para o negócio, evitando sobrecarregar as equipes com muitas exigências simultâneas. Os desenvolvedores precisam compreender os motivos das mudanças e participar das decisões.

Relação com os pontos negativos da engenharia de dados

O caso evidencia problemas que também podem ocorrer em ambientes de engenharia de dados e infraestrutura:

- Complexidade operacional: múltiplas contas, permissões e configurações aumentam o esforço de trabalho.
- Documentação inadequada: informações desatualizadas dificultam a utilização e a manutenção das plataformas.
- Burocracia na governança: regras excessivas ou impostas sem contexto podem prejudicar a produtividade.
- Custos e falta de rastreabilidade: a ausência de tags dificulta identificar responsáveis e atribuir custos aos recursos.
- Resistência à adoção: mudanças que não consideram as necessidades dos usuários podem gerar frustração e aumentar a demanda por suporte.

4. DEFESA TÉCNICA DA TECNOLOGIA

Resistência Cultural e Atrito na Adoção

De fato, a apresentação que trouxe o argumento que fala sobre ser um problema organizacional com atrito está correto, com Da Paolis, o entrevistado da notícia recente feita pela infoQ sobre plataformas que ajudam desenvolvedores, descrevendo a mudança como “bagunçada no meio”, entretanto ele também adiciona que “apesar disso, vale a pena no fim.” para cuidar de problemas semelhantes ao Shadow IT, é possível utilizar a metodologia dele, em que ele diz que “transparência, empatia e implantação incremental são tão importantes quanto a tecnologia em si”, e que “se equipes verem conformidade como algo que ajuda eles a se moverem mais rápido e mais seguro, apoiados por detecção clara e guardrails sensatos, adoção segue naturalmente.”

Com isso, é possível concluir que uma comunicação clara que utiliza a experiência pessoal do desenvolvedor para considerar implementações de funcionalidades ausentes que iriam ser cobertas por Shadow IT, e trazer incentivos por meio de feedback e ferramentas relevantes e interessantes para os desenvolvedores.

Golden Cages

Primeiramente eu gostaria de distinguir que a apresentação em si já traz um ponto de importância, que **Golden paths e golden cages são diferentes**. um golden path tem o risco de se tornar em um golden cage, mas não automaticamente se transforma em um, especialmente com implementação apropriada.

Já se foi feito o ponto durante o aprofundamento do artigo científico de Anjum (2026) sobre o tópico, que a diferença crucial entre os dois é a **Restrição do desenvolvedor e seu desenvolvimento**. Se a decisão final sempre for do desenvolvedor, com ferramentas para trabalhar em níveis de abstração menores ou sair do padrão quando necessário para o desenvolvimento, um golden cage não tem a capacidade de se manifestar.

A plataforma tendo restrições por meio de falta de funcionalidades durante o período transicional dela é algo discutido em mais profundidade durante o tópico de atrito na Adoção, mas é importante mencionar que de acordo com a conclusão feita por Anjum sobre um estudo de developer onboarding feito por Jasper e Green no google, atritos futuros como o developer onboarding tiveram grandes benefícios graças à implementação de golden paths, mostrando explicitamente que ao longo

termo que os golden paths reduzem atrito ao invés de ampliar ele.

5. Réplica do grupo apresentador

A Engenharia de Plataformas pode, de fato, aumentar a complexidade quando é mal implementada. O contra-argumento apresentado pelo outro grupo destaca que uma Internal Developer Platform pode introduzir novas interfaces, configurações e abstrações para tarefas que antes eram mais simples, aumentando a carga cognitiva dos desenvolvedores.

A própria literatura reconhece esse risco, mas trata esse cenário como um problema de implementação, e não como uma característica inevitável da Engenharia de Plataformas.

No artigo de Anjum (2026), um dos pontos discutidos é o risco de criar abstrações e funcionalidades antes de entender quais problemas realmente precisam ser resolvidos. A ideia de uma plataforma interna não é concentrar o maior número possível de ferramentas e processos, mas oferecer apenas as capacidades que realmente reduzem atritos no trabalho dos desenvolvedores. Essa lógica se aproxima do conceito de uma plataforma mínima e evolutiva: primeiro identifica-se um problema real, depois cria-se uma solução para ele e, só então, a plataforma cresce conforme novas necessidades aparecem. Isso ajuda a evitar o chamado over-engineering e reduz a chance de a IDP se transformar em apenas mais uma camada de complexidade.

Esse ponto também aparece no trabalho de Ciancarini et al. (2025), que apresenta uma Internal Developer Platform auto-hospedada, de código aberto e estruturada com arquitetura baseada em microsserviços. A proposta mostra que uma plataforma pode ser construída de forma modular e extensível, permitindo que novas capacidades sejam adicionadas conforme necessário, em vez de concentrar tudo em uma estrutura rígida desde o início. Assim, a existência de uma camada adicional não significa automaticamente aumento de complexidade. O resultado depende principalmente do que essa camada substitui. Se ela transforma vários processos manuais, consultas de documentação, solicitações para outras equipes e configurações repetitivas em um fluxo único e automatizado, existe uma redução da complexidade percebida pelo desenvolvedor.

Outro ponto importante do contra-argumento é a falta de evidências empíricas sobre Engenharia de Plataformas. O próprio Anjum confirma que a área ainda possui uma produção acadêmica limitada quando comparada ao volume de conteúdo produzido pela indústria. O estudo analisou 88 fontes e identificou que apenas uma pequena parte delas vinha de venues acadêmicos de maior relevância tendo Platform Engineering como foco principal. Essa limitação é importante e não deve ser ignorada.

Porém, a falta de evidências suficientes não significa que existam evidências provando que a abordagem não funciona. O que existe atualmente é uma base científica ainda em amadurecimento, com níveis diferentes de evidência para cada prática. Algumas capacidades possuem resultados mais concretos, enquanto outras ainda são sustentadas principalmente por estudos de caso, experiências organizacionais e literatura da indústria.

O próprio trabalho original já reconhecia essa situação ao destacar a escassez de pesquisas empíricas, a influência da literatura industrial e a falta de métricas consolidadas para avaliar plataformas internas. Portanto, a conclusão mais adequada não é afirmar que Engenharia de Plataformas está totalmente comprovada, mas também não é afirmar que ela não funciona. O mais correto é entender que seus resultados dependem do contexto e precisam ser medidos dentro das próprias organizações.

Nesse sentido, a Developer Experience se torna um elemento importante para avaliar se uma plataforma realmente está cumprindo seu objetivo. Métricas técnicas, como tempo de entrega, frequência de deploy ou estabilidade, não deveriam ser utilizadas isoladamente. Elas precisam ser complementadas por informações sobre satisfação dos desenvolvedores, carga cognitiva, facilidade de uso, tempo de onboarding e utilização efetiva das funcionalidades da plataforma. O estudo de Greiler, Storey e Noda (2023), já utilizado no trabalho inicial, reforça essa ideia ao relacionar Developer Experience a produtividade, satisfação, engajamento e eficiência no trabalho.

Assim, os próprios pontos levantados pelo contra-argumento acabam reforçando uma visão mais cuidadosa da Engenharia de Plataformas. A abordagem não deve ser aplicada como uma solução universal nem adotada apenas porque está em destaque na indústria. Ela precisa partir de problemas concretos, crescer de forma gradual e ser avaliada continuamente com base nos resultados percebidos pelos desenvolvedores e pela organização.

Inclusive, a própria conclusão do grupo de oposição se aproxima bastante dessa interpretação. Eles afirmam que os riscos apresentados decorrem principalmente de implementações complexas, rígidas ou desalinhadas, e recomendam adoção baseada em problemas reais, Developer Experience e relação entre custo e benefício. Dessa forma, a pesquisa complementar permite mostrar que os problemas apresentados não necessariamente invalidam a Engenharia de Plataformas. Na verdade, eles representam justamente os principais cuidados que precisam ser considerados para que uma plataforma interna consiga atingir seu objetivo de reduzir complexidade, melhorar a experiência dos desenvolvedores e organizar melhor os processos de desenvolvimento.

Referências

ANJUM, Mateen Ali. **Platform engineering and internal developer portals: a multivocal literature review**. *Frontiers in Computer Science*, v. 8, 2026. DOI: 10.3389/fcomp.2026.1814498.

CIANCARINI, Paolo; GIANCARLO, Raffaele; GRIMAUDO, Gennaro; MISSIROLI, Marcello; XIA, Tian Cheng. **The design and realization of a self-hosted and open-source agile internal development platform**. *IEEE Access*, v. 13, p. 79516–79533, 2025. DOI: 10.1109/ACCESS.2025.3564141.

GREILER, Michaela; STOREY, Margaret-Anne; NODA, Abi. **An actionable framework for understanding and improving developer experience**. *IEEE Transactions on Software Engineering*, v. 49, n. 4, p. 1411–1425, 2023. DOI: 10.1109/TSE.2022.3175660.

LINDERS, Ben. **Achieving compliance as a platform engineering team by helping developers**. *InfoQ*, 23 jul. 2026. Acesso em: 30 set. 2026.